Haqdaar.ai: A Multimodal, Deterministic Eligibility Matcher for Government Welfare Schemes

Project Proposal for UCS503P
Submitted to: Paramveer Sidhu
Thapar Institute of Engineering and Technology

Dibyanshu Samal, CSED* Gunagye Jain, CSED† Yash Jagwan, CSED‡

August 16, 2026

1 Higher-order goal

. . . secondary framing

This project aims to improve civic accessibility by building **Haqdaar.ai**, a multimodal - voice- and text-driven - eligibility matcher for Indian government welfare schemes. The name *Haqdaar* (Hindi/Urdu for "one who is rightfully entitled") replaces the project's earlier working name to make the mission legible to citizens: helping people claim what they are already entitled to, not merely browse a list of schemes. The immediate engineering objective is to translate complex legal eligibility criteria into a measurable, deterministic software solution driven by natural-language input, whether typed or spoken.

2 Time-to-value

. . . primary heuristic

- We will deliver meaningful increments quickly by prototyping the core data-extraction and SQL matching workflow first, validating it end-to-end before layering on the voice interface.
- We will prioritize fast feedback over perfection with weekly iterations, bringing up the Next.js/NestJS stack with CI-backed automation early.
- Rather than seed the database with a small, hand-picked set of 15–20 schemes, we commit from the first iteration to a scraper pipeline against `myscheme.gov.in`, so even early testing happens against a realistic corpus of **150+ schemes** instead of a toy sample.
- Success is defined by what the first iteration delivers - typed-input form, deterministic matcher, scraped corpus - then improved with voice input based on user testing.

3 Problem Statement

Citizens face massive information asymmetry when determining which government schemes apply to them. Common pain points include:

- **High fragmentation:** thousands of schemes exist across central and state levels, with no unified discovery mechanism.

*Roll No: 1024030023, Email: dsamal_be24@thapar.edu
†Roll No: 1024030019, Email: gjain_be24@thapar.edu
‡Roll No: 1024030018, Email: yjagwan_be24@thapar.edu

- **Language and literacy barriers:** complex legal prose makes self-assessment difficult for the average citizen.
- **AI unreliability:** existing LLM-based solutions hallucinate, causing wrongful exclusions (false negatives) on eligibility.
- **Single-mode input risk:** voice-only excludes users in noisy or shared settings and has no recovery path for a bad transcription; text-only excludes citizens uncomfortable typing.

Why this matters now: Bridging this gap needs an interface citizens can either speak to, in their own language, or type into directly - backed by an engine that guarantees 100% deterministic, auditable results rather than AI guesswork.

4 Proposed Solution *... Software Proposition*

4.1 Overview

Build a web-based "Haqdaar.ai" system with the following components:

- **Multimodal input UI:** a browser interface offering both a typed form and voice dictation for the same profile fields. A spoken answer lands in an editable text box rather than being consumed silently, so users can switch freely and always correct a mis-transcription by hand.
- **LLM Extraction Layer:** an isolated LLM (e.g., Llama 3) strictly converts spoken transcripts into structured JSON fields, never deciding eligibility. Typed input reaches the profile directly, bypassing this layer.
- **Deterministic Matching Engine:** a PostgreSQL JSONB logic layer that evaluates the profile - however entered - against normalized scheme rules.
- **Conversational Loop:** the system surfaces the "next best question" (spoken in voice mode, shown on-screen in text mode) to resolve unknown criteria.

4.2 Core workflow

1. Citizen either types their profile or speaks it aloud (e.g., "I am a 42-year-old farmer from UP"); both paths write into the same profile object.
2. If spoken, the system transcribes and extracts structured fields (Age: 42, Occupation: Farmer, State: UP) into the same editable fields the typed path uses, for review before use.
3. System matches known fields deterministically against the database, flagging remaining variables as `UNKNOWN`.
4. System asks the optimal follow-up question back - by voice or on-screen text, matching the user's mode - to narrow down the scheme list.

4.3 Operational constraints for an educational, tier-2/3 setting

- Keep the solution usable on standard mobile and desktop browsers via responsive web design and native MediaRecorder APIs.
- Keep the typed form fully functional on its own, so voice is strictly additive: a user who never speaks a word can still complete a profile and get a match.
- Avoid LLM hallucinations by restricting the AI to data extraction only; focus on workflow, normalization, and correctness.

- Design for deployability using managed containerized databases (PostgreSQL) and standard web hosting.

5 Solution Approach *... Engineering focus*

This is an engineering course project, so we focus on scalable data structures, deterministic workflows, and corpus-acquisition engineering rather than novel LLM training.

5.1 Data extraction and corpus acquisition *... non-research, pushing for scale*

Both live extraction (voice) and one-time corpus ingestion (scraper) are strictly schema-driven:

- Use Zod validation so LLM output - from either the voice extractor or the scheme-scraping normalizer - precisely matches the database schema.
- Ground every scraped numeric bound (income caps, age limits) back against the source prose before insertion, so no invented figure can silently exclude an eligible citizen; relax incalculable conditionals to a wildcard status flagged for manual review rather than dropped or guessed at.
- A one-time, later periodic, batch scraper walks the paginated listing on `myscheme.gov.in` to reach a target corpus of **150+ schemes** - chosen because 15–20 hand-typed schemes would not meaningfully demonstrate the fragmentation problem in Section 3.
- Never let an extracted or scraped field auto-submit without being visible and editable to a human - the citizen for voice input, the team for scraped scheme rules.

5.2 Web architecture *... web-based solution*

A typical 3-tier web architecture:

- **Frontend:** Next.js responsive UI with a single shared profile state consumed by both the typed form and the voice console, so neither path is a second system.
- **Backend API:** NestJS handling provider selection (STT/TTS), the conversational turn loop, and the matching endpoint.
- **Database:** PostgreSQL storing structured rules in JSONB columns for single-query evaluations, alongside the raw scraped prose for each scheme so any rule can be audited against its source.

5.3 CI/CD and fast delivery enabler

CI/CD builds and runs backend unit tests (especially matching-engine assertions) on every change, produces repeatable database migrations and seed/scrape scripts, and enables safe, frequent releases of incremental features.

6 Evaluation Criterion *... measurable and attributable*

We define success using measurable metrics tied to the core workflow.

6.1 Primary evaluation metric

Turn Latency (voice) / Response Latency (text): time between the user finishing an input - spoken or typed - and the system returning the next question or match result.

- Target: median latency ≤ 2 seconds during pilot window, for either mode.
- Attribution: measured via timestamps spanning STT transcription and LLM extraction (voice only) and SQL matching (both modes).

6.2 Secondary evaluation metrics

- **Extraction accuracy:** 0% hallucinated fields during LLM data extraction on the voice path (verified via benchmark scripts).
- **Matching speed:** sub-100ms execution time for the SQL matching function across the full corpus.
- **Corpus coverage:** unique schemes successfully scraped, normalized, and passing schema validation from `myscheme.gov.in` (target ≥ 150).
- **Reliability:** seamless degradation to browser-native TTS/typed-form input if primary AI APIs are rate-limited or unavailable.

6.3 Pilot validation plan *... high-level*

- Test the pipeline against the full scraped database of 150+ schemes, not a curated subset.
- Recruit pilot testers to complete one profile entirely via typing and one entirely via voice, confirming both paths are independently sufficient.
- Compare accuracy and latency across LLM providers (e.g., Groq vs. a local mock) during the iteration window.

7 Scalability *... with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** support growth beyond the 150+ scheme baseline without degradation, via GIN indexing on the JSONB eligibility column, a single stateless PostgreSQL query for matching, and a scraper pipeline decoupled from the live conversational API.
2. **Operationally deployable (practically):** run on common web hosting - a managed PostgreSQL database (e.g., Supabase/Render), containerized API deployment, and Vercel-style edge deployment for the Next.js frontend.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a web-based project: Next.js (React) and NestJS as web frameworks; PostgreSQL with Prisma as the managed relational database; a speech-to-text/text-to-speech provider (e.g., Sarvam AI) and an LLM provider (e.g., Groq) for extraction only; Playwright for one-time and periodic corpus scrapes of `myscheme.gov.in`; and a test framework for backend assertions and prompt evaluations. We will use these mature components to focus on workflow design, schema enforcement, corpus scale, and latency optimization rather than inventing new infrastructure.

9 Project scope and deliverables *. . . iteration-friendly*

9.1 Initial deliverable *. . . first iteration*

- Database schema initialization with JSONB rules and the `match_schemes()` SQL function.
- Scraper pipeline against `myscheme.gov.in` targeting 150+ schemes, with prose-grounded validation before insert.
- Typed applicant profile form - the multimodal safety net, always available standalone.
- Basic result cards showing PASS/FAIL/UNKNOWN reasoning; CI with build and matcher unit tests.

9.2 Subsequent deliverables

- Microphone integration and STT provider routing, writing into the same editable fields the typed path uses.
- LLM profile-extraction prompt and JSON schema validation.
- Text-to-Speech playback of the "Next Question," with on-screen text shown in parallel for accessibility.
- Scheduled re-scrape automation to keep the scheme corpus current as policy changes.

10 Risks and mitigations

- **LLM hallucinations and silent transcription errors:** mitigated by strict separation of concerns - the LLM only formats user input while deterministic SQL rules handle eligibility math - and by never auto-submitting a voice-extracted field without surfacing it in the same editable box the typed path uses.
- **API rate limits/latency:** mitigated by strict timeout limits and graceful downgrades to browser-native `SpeechSynthesis` and the typed form.
- **Scrape coverage and staleness:** benefit amounts and eligibility text change with policy; mitigated by keeping scraped prose alongside the structured rule for auditability and scheduling periodic re-scrapes.

11 Summary

This project proposes Haqdaar.ai, a deployable, multimodal web platform that solves the civic information gap surrounding government welfare schemes. It meets the course criteria by emphasizing time-to-value, implementing measurable evaluation criteria for latency, accuracy, and corpus coverage, and ensuring scalability. Two commitments distinguish it from a narrower version of the same idea: offering *both* voice and text input, so a transcription error is always recoverable; and sourcing eligibility data from a scraped, production-scale corpus of 150+ real schemes rather than a small hand-typed sample, so the tool actually confronts the fragmentation problem it exists to solve. It remains focused on sound software engineering - data validation, deterministic state management, and data-pipeline scale - rather than purely research-driven generative AI.